



SJB INSTITUTE OF TECHNOLOGY

#67, B G S HEALTH AND EDUCATION CITY
Kengeri, Bengaluru-560060.



DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

DIGITAL SIGNAL PROCESSING (IPCC21EC42)

MODULE-5

DIGITAL SIGNAL PROCESSORS

Prepared By:
Mr. Rahul Rai
Asst. Prof., Dept. of ECE

Module-5

DIGITAL SIGNAL PROCESSORS

Syllabus

DSP Architecture, DSP Hardware Units, Fixed point format, Floating point Format, IEEE Floating point formats, Fixed point digital signal processors, Floating point processors, FIR and IIR filter implementations in Fixed point systems.

Unlike microprocessors and microcontrollers, digital signal (DS) processors have special features that require operations such as fast Fourier transform (FFT), filtering, convolution and correlation, and real-time sample-based and block-based processing. Therefore, DS processors use a different dedicated hardware architecture.

Von Neumann and Harvard Architectures

- A **Von Neumann processor** (Figure 1) contains **a single shared memory for programs and data, a single bus for memory access, an arithmetic unit, and a program control unit.**
- The processor proceeds in a serial fashion in terms of fetching and execution cycles. This means that the central processing unit (CPU) fetches an instruction from memory and decodes it to figure out what operation to do, then executes the instruction.
- The instruction (in machine code) has two parts: the **opcode** and the **operand**.
- The **opcode** specifies what the operation is, that is, tells the CPU what to do.
- The **operand** informs the CPU what data to operate on. These instructions will modify memory, or input and output (I/O). After an instruction is completed, the cycles will resume for the next instruction. One instruction or piece of data can be retrieved at a time.
- Since the processor proceeds in a serial fashion, it causes most units to stay in a wait state.
- The **Von Neumann architecture** operates the cycles of fetching and execution by fetching an instruction from memory, decoding it via the program control unit, and finally executing instruction.
- When execution requires data movement that is, data to be read from or written to memory the next instruction will be fetched after the current instruction is completed.
- The Von Neumann-based processor has this bottleneck mainly due to the use of **a single, shared memory for both program instructions and data.**
- Increasing the speed of the bus, memory, and computational units can improve speed, but not significantly.

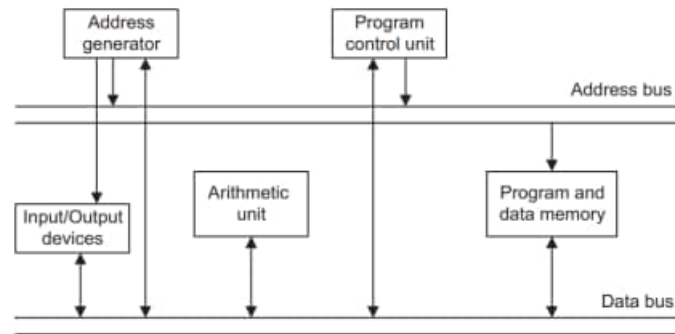


Figure 1: General microprocessor based on Von Neumann architecture.

- To accelerate the execution speed of digital signal processing, DS processors are designed based on the Harvard architecture (Figure 2), which originated from the Mark 1 relay-based computers built by IBM in 1944 at Harvard University.
- The DS processor has **two separate memory spaces**. One is dedicated for the program code, while the other is employed for data.
- Hence, to accommodate two memory spaces, **two corresponding address buses and two data buses** are used.
- In this way, the program memory and data memory have their own connections to the program memory bus and data memory bus, respectively.
- This means that the Harvard processor **can fetch the program instruction and data in parallel at the same time**, the former via the program memory bus and the latter via the data memory bus.
- There is an additional unit called a **multiplier and accumulator** (MAC), which is the dedicated hardware used for the digital filtering operation.
- The last additional unit, the **shift unit**, is used for the scaling operation for fixed-point implementation when the processor performs digital filtering.

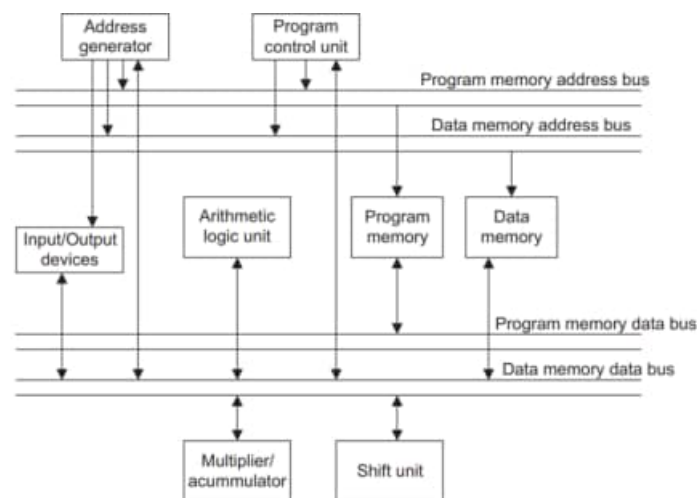


Figure 2: Digital signal processors based on the Harvard architecture.

- Comparison of executions of the two architectures
- The Von Neumann architecture generally has the execution cycles described in Figure 3.
- The *fetch cycle* obtains the opcode from the memory, and the control unit will decode the instruction to determine the operation.
- Next is the *execute cycle*. Based the decoded information, execution will modify the content of the register or the memory.
- Once this is completed, the process will fetch the next instruction and continue.
- The processor operates *one instruction at a time in a serial fashion*.
- To improve the speed of the processor operation, the Harvard architecture takes advantage of a common DS processor, in which one register holds the filter coefficient while the other register holds the data to be processed, as depicted in Figure 4.
- As shown in Figure 4, the execute and fetch cycles are overlapped.
- We call this the ***pipelining operation***.
- The DS processor performs one execution cycle while also fetching the next instruction to be executed. Hence, the processing speed is dramatically increased.

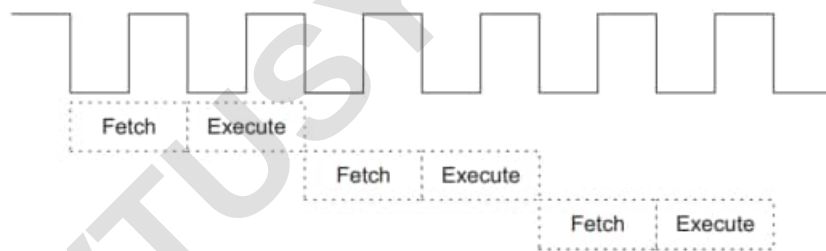


Figure 3: Execution cycle based on the Von Neumann Architecture.

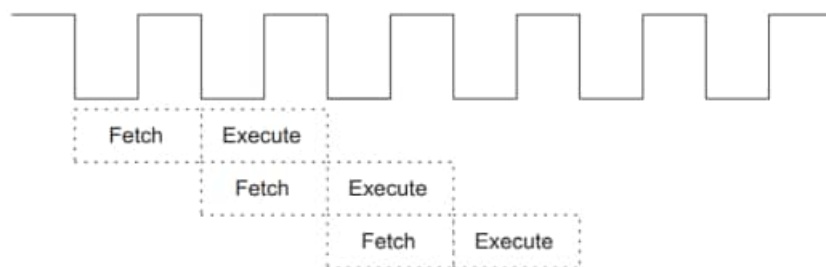


Figure 4: Execution cycle based on the Harvard Architecture.

Application:

- The Harvard architecture is preferred for all DS processors due to the requirements of most digital signal processing (DSP) algorithms, such as filtering, convolution, and FFT, which need repetitive arithmetic operations, including multiplications, additions, memory access, and heavy data flow through the CPU.
- For the other applications, such as those dependent on simple microcontrollers with less of a timing requirement, the Von Neumann architecture may be a better choice, since it offers much less silica area and is thus less expensive.

DIGITAL SIGNAL PROCESSOR HARDWARE UNITS**1) Multiplier and Accumulator (MAC)**

- As compared with the general microprocessors based on the Von Neumann architecture, the DS processor uses the MAC, a special hardware unit for enhancing the speed of digital filtering.
- This is dedicated hardware, and the corresponding instruction is generally referred to as a MAC operation.
- The basic structure of the MAC is shown in Figure 5.
- As shown in Figure 5, in a typical hardware MAC, the multiplier has a pair of **input registers**, each holding the 16-bit input to the multiplier.
- The result of the multiplication is **accumulated** (added) in the 32-bit accumulator unit.
- The **result register** holds the double precision data from the accumulator.

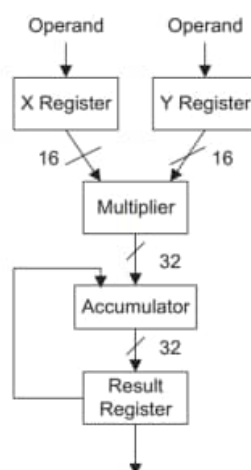


Figure 5: The multiplier and accumulator (MAC) dedicated to DSP.

2) Shifters

- In digital filtering, to prevent overflow, a scaling operation is required.
- A simple scaling-down operation shifts data to **the right**, while a scaling-up operation shifts data to **the left**.
- Shifting data to the right is the same as dividing the data by 2 and truncating the fraction part; shifting data to the left is equivalent to multiplying the data by 2.

E.g. for a 3-bit data word $011_2 = 3_{10}$, shifting 011 to the right gives $001_2 = 1$, that is, $3/2=1.5$, and truncating 1.5 results in 1.

Shifting the same number to the left, we have $110_2 = 6_{10}$, that is, $3 \times 2 = 6$.

- The DS processor often shifts data by several bits for each data word.
- To speed up such operation, the special hardware shift unit is designed to accommodate the scaling operation, as depicted in Figure 2.

3) Address Generators

- The DS processor generates the addresses for each datum on the data buffer to be processed.
- A special hardware unit for *circular buffering* is used.
- Figure 9.6 describes the basic mechanism of circular buffering for a buffer having eight data samples.
- In circular buffering, a pointer is used and always points to the newest data sample, as shown in Figure 9.6.
- After the next sample is obtained from analog-to-digital conversion (ADC), the data will be placed at the location of $x(n-7)$ and the oldest sample is pushed out.
- Thus, the location for $x(n-7)$ becomes the location for the current sample.
- The original location for $x(n)$ becomes a location for the past sample of $x(n-1)$. The process continues. Only one location is updated.
- The circular buffer acts like a first-in/first-out (FIFO) buffer, but each datum on the buffer does not have to be moved.
- Figure gives a simple illustration of the 2-bit circular buffer. In the figure, there is data flow to the ADC (a, b, c, d, e, f, g, .) and a circular buffer initially containing a, b, c, and d.
- The pointer specifies the current data of d, and the equivalent FIFO buffer is shown on the right side with the current data of d at the top of the memory.
- Like finite impulse response (FIR) filtering, the data buffer size can reach several hundreds. Hence, using the circular buffer will significantly enhance the processing speed.

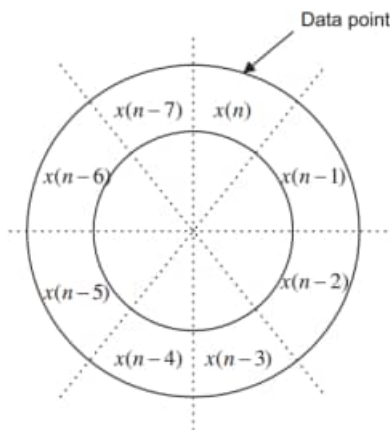


Figure 5: Illustration of circular buffering.

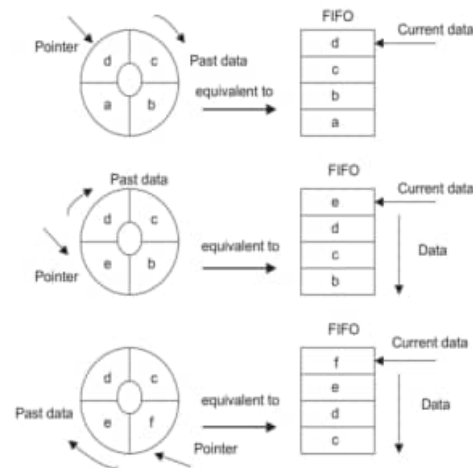


Figure 6: Circular buffer and equivalent FIFO.

FIXED-POINT AND FLOATING-POINT FORMATS

- In order to process real-world data, we need to select an appropriate DS processor, as well as a DSP algorithm or algorithms for a certain application.
- Whether a DS processor uses a fixed- or floating-point method depends on how the processor's CPU performs arithmetic.
- A **fixed-point DS processor** represents data in **2's complement integer format** and manipulates data using **integer arithmetic**, while a **floating-point processor** represents number using a **mantissa (fractional part)** and an **exponent** in **addition to the integer format** and operates data using **floating-point arithmetic**.
- Since the fixed-point DS processor operates using the integer format, which represents only a very narrow **dynamic range** of the integer number, a problem such as **overflow** of data manipulation may occur. Hence, we need to spend much more coding effort to deal with such a problem.
- As we shall see, we may use floating-point DS processors, which offer a **wider dynamic range** of data, so that coding becomes much easier. (Dynamic range is the ratio between the largest and smallest values that a certain quantity can assume.)
- However, the floating-point DS processor contains **more hardware units** to handle the integer arithmetic and the floating-point arithmetic; hence it is more expensive and slower than fixed-point processors in terms of instruction cycles. It is usually a choice for prototyping or proof-of-concept development.

Fixed-Point Format

- Considering a 3-bit 2's complement, we can represent all the decimal numbers shown in Table A.

Table A. 3-bit 2's complement representation

DECIMAL NUMBER	TWO'S COMPLIMENT
3	011
2	010
1	001
0	000
-1	111
-2	110
-3	101
-4	100

- As we see, a 3-bit 2's complement number system has a dynamic range from -4 to 3, which is very narrow. Since the basic DSP operations include multiplications and additions, results of operation can cause **overflow problems**. Overflow occurs when the result magnitude is greater than the limits that can be held destination location.

Given

a. $2 \times (-1)$

b. $2 \times (-3)$

operate each expression using 2's complement.

Solution:

a.

$$\begin{array}{r}
 010 \\
 \times 001 \\
 \hline
 010 \\
 000 \\
 + 000 \\
 \hline
 00010
 \end{array}$$

The 2's complement of 00010 = 11110. Removing two extended sign bits 1 gives 110. The answer is 110 (-2), which is within the system.

b.

$$\begin{array}{r}
 010 \\
 \times 011 \\
 \hline
 010 \\
 010 \\
 + 100 \\
 \hline
 00110
 \end{array}$$

The 2's complement of 00110 = 11010. Removing two extended sign bits leaves 010. Since the binary number 010 is 2, which is not (-6) as what we expect, overflow occurs; that is, the result of the multiplication (-6) is out of our dynamic range (-4 to 3).

- Decimal values as fractional numbers, to obtain the fractional binary 2's complement system shown in Table below.
- To become familiar with the fractional binary 2's complement system, let us convert a positive fraction number $\frac{3}{4}$ and a negative fraction number $-\frac{1}{4}$ in decimals to their 2's complements.

Since $\frac{3}{4} = 0 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2}$ its 2's complement is **011**.

And $\frac{1}{4} = 0 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2}$ its 2's complement is **001**.

For $-\frac{1}{4}$ taking two's complement $(110 + 1) = \mathbf{111}$.

Table B. 3-bit 2's complement system using Fractional Representation.

DECIMAL NUMBER	DECIMAL FRACTION	TWO'S COMPLIMENT
3	3/4	0.11
2	2/4	0.10
1	1/4	0.01
0	0	0.00
-1	-1/4	1.11
-2	-2/4	1.10
-3	-3/4	1.01
-4	-4/4 = -1	1.00

Now let us focus on the fractional binary 2's complement system.

The data are normalized to the fractional range from -1 to $1 - 2^2 = 3/4$.

- When we carry out multiplications with two fractions, the result should be a fraction, so that multiplication overflow can be prevented. Let us verify the multiplication $(0.10)(1.01)$, which is the overflow case in previous Example.
- We first multiply two positive numbers.

$$\begin{array}{r}
 0.10 \\
 \times 0.11 \\
 \hline
 010 \\
 010 \\
 + 100 \\
 \hline
 0.0110
 \end{array}$$

The 2's complement of $0.0110 = \mathbf{1.1010}$

The answer in decimal form should be

$$1.1010 = (-1) \times (0.0110)_2 = -(0 \times (2)^{-1} + 1 \times (2)^{-2} + 1 \times (2)^{-3} + 0 \times (2)^{-4}) = -\frac{3}{8}$$

This number is correct, as we can verify from Table 9.2, that is, $\left(\frac{2}{4} \times \left(-\frac{3}{4}\right)\right) = -\frac{3}{8}$.

If we truncate the last two least significant bits to keep the 3-bit binary number, we have an approximate answer:

$$1.10 = (-1) \times (0.01)_2 = -(0 \times (2)^{-1} + 1 \times (2)^{-2}) = -\frac{1}{2}$$

Truncation error occurs. The error should be bounded by $2^{-2} = \frac{1}{4}$. We can verify that

$$|-1/2 - (-3/8)| = 1/8 < 1/4$$

With such a scheme, we can avoid overflow due to multiplication but cannot prevent overflow due to addition. Consider the addition example

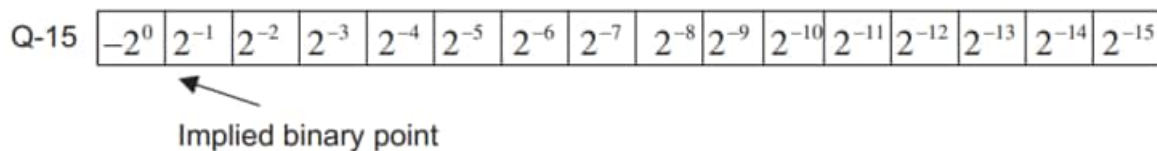
$$\begin{array}{r}
 0.11 \\
 + 0.01 \\
 \hline
 1.00
 \end{array}$$

where the result 1.00 is a negative number. Adding two positive fractional numbers yields a negative number. Hence, overflow occurs.

Q- Format

- Q-format number representation is the most common one used in fixed-point DSP implementation.
- The Q number represents the number of **fractional bits**. Q-15 has 15 fractional bits.

It is defined in Figure below.



- Q-15 means that the data are in a sign magnitude form in which there are **15 bits for magnitude** and **one bit for sign**. Note that after the sign bit, the dot shown in Figure implies the binary point.
- The number is normalized to the fractional range from -1 to 1.
- The range is divided in to 2^{16} intervals, each with a size of 2^{-15} . The most negative number is -1, while the most positive number is $1-2^{-15}$.
- Any result from multiplication is within the fractional range of -1 to 1.

Table C: Conversion process of Q-15 Representation

Number	Product	Carry
0.560123 x 2	1.120246	1 (MSB)
0.120246 x 2	0.240492	0
0.240492 x 2	0.480984	0
0.480984 x 2	0.961968	0
0.961968 x 2	1.923936	1
0.923936 x 2	1.847872	1
0.847872 x 2	1.695744	1
0.695744 x 2	1.391488	1
0.391488 x 2	0.782976	0
0.782976 x 2	1.565952	1
0.565952 x 2	1.131904	1
0.131904 x 2	0.263808	0
0.263808 x 2	0.527616	0
0.527616 x 2	1.055232	1
0.055232 x 2	0.110464	0 (LSB)

MSB, most significant bit; LSB, least-significant bit.

Let us study the following examples to become familiar with Q-format number representation.

Example:

Find the signed Q-15 representation for the decimal number 0.560123.

Solution:

The conversion process is illustrated using **Table C**. For a positive fractional number, we multiply the number by 2 if the product is larger than 1, carry bit 1 as a most significant bit (MSB), and copy the fractional part to the next line for the next multiplication by 2; if the product is less than 1, we carry bit 0 to MSB. The procedure continues to collect all 15 magnitude bits.

We yield the Q-15 format representation as

$$0.100011110110010$$

Since we only use 16 bits to represent the number, we may lose accuracy after conversion. Like quantization, truncation error is introduced. However, this error should be less than the interval size, in this case, $2^{-15} = 0.0000305017$. We shall verify this in Example 9.5. An alternative method of conversion is to convert a fraction, let's say $\frac{3}{4}$ to Q-2 format, multiply it by 2^2 , and then convert the truncated integer to its binary, that is,

$$(3/4) \times 2^2 = 3 = 011_2$$

In this way, it follows that

$$(0.560123) \times 2^{15} = 18,354 \quad 100\ 0111\ 1011\ 0010$$

Converting 18,354 to its binary representation will achieve the same answer. The next example illustrates the signed Q-15 representation for a negative number.

EXAMPLE 9.3

Find the signed Q-15 representation for the decimal number -0.160123 .

Solution:

Converting the Q-15 format for the corresponding positive number with the same magnitude using the procedure described in Example 9.2, we have

$$0.160123 = 0.001010001111110$$

Then after applying 2's complement, the Q-15 format becomes

$$-0.160123 = 1.110101110000010$$

Alternative method: Since $(-0.160123) \times 2^{15} = -5,246.9$, converting the truncated number $-5,246$ to its 16-bit 2's complement yields 1110101110000010.

EXAMPLE 9.4

Convert the Q-15 signed number 1.110101110000010 to the decimal number.

Solution:

Since the number is negative, applying the 2's complement yields

$$0.001010001111110$$

Then the decimal number is

$$-(2^{-3} + 2^{-5} + 2^{-9} + 2^{-10} + 2^{-11} + 2^{-12} + 2^{-13} + 2^{-14}) = -0.160095$$

EXAMPLE 9.5

Convert the Q-15 signed number 0.100011110110010 to the decimal number.

Solution:

The decimal number is

$$2^{-1} + 2^{-5} + 2^{-6} + 2^{-7} + 2^{-8} + 2^{-10} + 2^{-11} + 2^{-14} = 0.560120$$

As we know, the truncation error in Example 9.2 is less than $2^{-15} = 0.000030517$. We verify that the truncation error is bounded by

$$|0.560120 - 0.560123| = 0.000003 < 0.000030517$$

Note that the larger the number of bits used, the smaller the truncation error that may accompany it.

Examples 9.6 and 9.7 are devoted to illustrating data manipulations in the Q-15 format.

EXAMPLE 9.6

Add the two numbers in Examples 9.4 and 9.5 in Q-15 format.

Solution:

Binary addition is carried out as follows:

$$\begin{array}{r} 1.110101110000010 \\ + 0.100011110110010 \\ \hline 10.011001100110100 \end{array}$$

Then the result is

$$0.011001100110100$$

This number in decimal form is

$$2^{-2} + 2^{-3} + 2^{-6} + 2^{-7} + 2^{-10} + 2^{-11} + 2^{-13} = 0.400024$$

EXAMPLE 9.7

This is a simple illustration of fixed-point multiplication.

Determine the fixed-point multiplication of 0.25 and 0.5 in Q-3 fixed-point 2's complement format.

Solution:

Since $0.25 = 0.010$ and $0.5 = 0.100$, we carry out binary multiplication as follows:

$$\begin{array}{r} 0.010 \\ \times 0.100 \\ \hline 0000 \\ 0000 \\ 0010 \\ + 0000 \\ \hline 0.001000 \end{array}$$

Truncating the least significant bits to convert the result to Q-3 format, we have

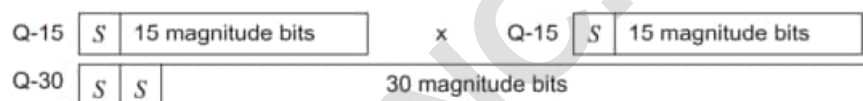
$$0.010 \times 0.100 = 0.001$$

Note that $0.001 = 2^{-3} = 0.125$. We can also verify that $0.25 \times 0.5 = 0.125$.

The **Q-format number** representation is a better choice than the 2's complement integer representation, it **can prevent multiplication overflow**.

But we need to be concerned with the following **problems**.

- 1) When converting a decimal number to its Q-N format, where N denotes the number of magnitude bits, we may lose accuracy due to the truncation error, which is bounded by the size of the interval, that is, 2^{-N} .
- 2) Addition and subtraction may cause overflow, where adding two positive numbers leads to a negative number, or adding two negative number yields a positive number; similarly, subtracting a positive number from a negative number gives a positive number, while subtracting a negative number from a positive number results in a negative number.
- 3) Multiplying two numbers in Q-15 format will lead to a Q-30 format, which has 31 bits in total. As in Example 9.7, the multiplication of Q-3 yields a Q-6 format, that is, 6 magnitude bits and a sign bit. In practice, it is common for a DS processor to hold the multiplication result using a double word size such as MAC operation, for multiplying two numbers in Q-15 format. In Q-30 format, there is one sign-extended bit. We may get rid of it by shifting left by one bit to obtain Q-31 format and maintaining the Q-31 format for each MAC operation.



- 4) Underflow can happen when the result of multiplication is too small to be represented in the Q-format. As an example, in a Q-2 system (No. of fractional bits = 2) multiplying 0.01×0.01 leads to 0.0001 . To keep the result in Q-2, we truncate the last two bits of 0.0001 to achieve 0.00 , which is zero. Hence, underflow occurs.

Floating-Point Format

- To increase the dynamic range of number representation, a floating-point format, which is similar to scientific notation, is used.
- The general format for floating-point number representation is given by

$$M \cdot 2^E$$

- where **M is the mantissa**, or fractional part in Q format, and **E is the exponent**. The mantissa and exponent are *signed numbers*. If we assign 12 bits for the mantissa and 4 bits for the exponent, the format looks like Figure 7.

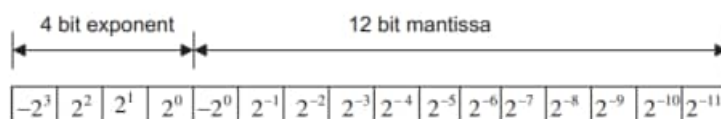


Figure 7: Floating point format.

- Since the 12-bit mantissa is limited to between -1 to +1, the number of bits assigned to the exponent controls the dynamic range.
- The bigger the number of bits designated to the exponent, the larger the dynamic range.
- The number of bits for the mantissa defines the interval in the normalized range; as shown in Figure 7, the interval size is 2^{-11} in the normalized range, which is smaller than the Q-15 format. However, when more mantissa bits are used, there will be a smaller interval size. Using the format in Figure 7, we can determine the most negative and most positive numbers as

$$\text{most negative number} = (1.00000000000)_2 \cdot 2^{0111_2} = (-1) \times 2^7 = -128.0$$

$$\text{most positive number} = (0.11111111111)_2 \cdot 2^{0111_2} = (1 - 2^{-11}) \times 2^7 = 127.9375$$

The smallest positive number is given by

$$\text{smallest positive number} = (0.00000000001)_2 \cdot 2^{1,000_2} = (2^{-11}) \times 2^{-8} = 2^{-19}$$

The exponent acts like a scale factor to increase the dynamic range of the number representation.

EXAMPLE 9.8

Convert each of the following decimal numbers to a floating-point number using the format specified in Figure 9.10.

- 0.1601230
- 20.430527

Solution:

a. We first scale the number 0.1601230 to $0.160123/2^{-2} = 0.640492$ with an exponent of -2 (other choices could be 0 or -1) to get $0.160123 = 0.640492 \times 2^{-2}$. Using 2's complement, we have $-2 = 1110$. Now we convert the value 0.640492 using the Q-11 format to get 010100011111. Cascading the exponent bits and the mantissa bits yields

$$1110010100011111$$

b. Since $-20.430527/2^5 = -0.638454$, we can convert it into the fractional part and exponent part as $-20.430527 = -0.638454 \times 2^5$. Note that this conversion is not particularly unique; the forms $-20.430527 = -0.319227 \times 2^6$ and $-20.430527 = -0.1596135 \times 2^7$ are still valid choices. Let us keep what we have now. Therefore, the exponent bits should be 0101.

Converting the number 0.638454 using the Q-11 format gives

$$010100011011$$

Using 2's complement, we obtain the representation for the decimal number -0.6438454 as

$$101011100101$$

Cascading the exponent bits and mantissa bits, we achieve

$$0101101011100100$$

Floating point arithmetic is more complicated. We must obey the rules for manipulating two floating point numbers.

Addition of two floating point numbers given as

$$x_1 = M_1 2^{E_1}$$

$$x_2 = M_2 2^{E_2}$$

It is required to make the exponents equal (here the greater one is chosen); this is called **aligning the exponents**. The floating-point sum is performed as follows:

$$x_1 + x_2 = \begin{cases} (M_1 + M_2 \times 2^{-(E_1-E_2)}) \times 2^{E_1}, & \text{if } E_1 \geq E_2 \\ (M_1 \times 2^{-(E_2-E_1)} + M_2) \times 2^{E_2}, & \text{if } E_1 < E_2 \end{cases}$$

For multiplication, given two properly normalized floating-point numbers.

$$x_1 = M_1 2^{E_1}$$

$$x_2 = M_2 2^{E_2}$$

where $0.5 \leq |M_1| < 1$ and $0.5 \leq |M_2| < 1$, the calculation can be performed as follows:

$$x_1 \times x_2 = (M_1 \times M_2) \times 2^{E_1+E_2} = M \times 2^E$$

The mantissas are multiplied while the exponents are added:

$$M = M_1 \times M_2$$

$$E = E_1 + E_2$$

EXAMPLE 9.9

Add the two floating point numbers obtained in Example 9.8:

$$1110\ 010100011111 = .640136718 \times 2^{-2}$$

$$0101\ 101011100101 = -0.638183593 \times 2^5$$

Solution:

Before addition, we change the first number so it has the same exponent as the second number, that is,

$$0101\ 000000001010 = 0.00500168 \times 2^5$$

Then we add the two mantissa numbers:

$$\begin{array}{r} 0.00000001010 \\ + 1.01011100101 \\ \hline 1.01011101111 \end{array}$$

The floating number is

$$0101\ 101011101111$$

We can verify the result by the following:

$$\begin{aligned} 0101\ 101011101111 &= -(2^{-1} + 2^{-3} + 2^{-7} + 2^{-11}) \times 2^5 = -0.633300781 \times 2^5 \\ &= -20.265625 \end{aligned}$$

EXAMPLE 9.10

Multiply the two floating-point numbers obtained in Example 9.8:

$$1110\ 010100011111 = .640136718 \times 2^{-2}$$

$$0101\ 101011100101 = -0.638183593 \times 2^5$$

Solution:

From the results in Example 9.8, we have the bit patterns for these two numbers as

$$E_1 = 1110, E_2 = 0101, M_1 = 010100011111, M_2 = 101011100101$$

Adding two exponents in 2's complement form leads to

$$E = E_1 + E_2 = 1110 + 0101 = 0011$$

which is +3, as we expected, since in the decimal domain $(-2) + 5 = 3$. As previously shown when introducing the multiplication rule, when multiplying two mantissas, we need to apply their corresponding positive values. If the sign for the final value is negative, then we convert it to its 2's complement form. In our example, $M_1 = 010100011111$ is a positive mantissa. However, $M_2 = 101011100101$ is a negative mantissa, since

the MSB is 1. To perform multiplication, we use 2's complement to convert M_2 to its positive value, 010100011011, and note that the multiplication result is negative. We multiply two positive mantissas and truncate the result to 12 bits to give

$$010100011111 \times 010100011011 = 001101010100$$

Now we need to add a negative sign to the multiplication result with the 2's complement operation. Taking the 2's complement, we have

$$M = 110010101100$$

Hence, the product is achieved by cascading the 4-bit exponent and 12-bit mantissa as

$$0011110010111100$$

Converting this number back to the decimal number, we verify the result to be

$$-0.408203125 \times 2^3 = -3.265625.$$

Overflow

During an operation, overflow will occur when a number is too large to be represented in the floating-point number system. Adding two mantissa numbers may lead to a number larger than +1 or less than -1; and multiplying two numbers causes the addition of their two exponents so that the sum of the two exponents could overflow. Consider the following overflow cases.

Case 1.

Add the following two floating-point numbers:

$$0111011000000000 + 0111010000000000$$

Note that the two exponents are the same and they are the biggest positive number in 4-bit 2's complement representation. We add two positive mantissa numbers as

$$\begin{array}{r} 0.1100000000 \\ + 0.1000000000 \\ \hline 1.0100000000 \end{array}$$

MSB is 1. The result for adding mantissa numbers is negative. Hence the **overflow occurs**.

Case 2: Multiply the following two numbers:

$$0111\ 011000000000 \times 0111\ 011000000000$$

Adding the two positive exponents gives $0111 + 0111 = 1000$ (negative, the overflow occurs)

Multiplying the two mantissa numbers gives

$$0.110000000000 \times 0.110000000000 = 0.100100000000 \text{ (This is OK!)}$$

Underflow

As we discussed before, underflow will occur when a number is too small to be represented in the number system. Let us divide the following two floating-point numbers:

$$1001\ 001000000000 \div 0111\ 010000000000$$

First, subtracting the two exponents leads to

$$1001(\text{negative}) - 0111(\text{positive}) = 1001 + 1001 = 0010 \text{ (positive; the underflow occurs)}$$

Then, dividing two mantissa numbers, it follows that

$$0.010000000000 \div 0.100000000000 = 0.100000000000 \text{ (OK!)}$$

However, in this case, the expected resulting exponent is -14 in decimal, which is too small to be presented in the 4-bit 2's complement system. Hence the underflow occurs.

Now that we understand the basic principles of the floating-point formats, we can next examine two floating-point formats of the Institute of Electrical and Electronics Engineers (IEEE).

IEEE Floating-Point Formats

1. Single Precision Format

IEEE floating-point formats are widely used in many modern DS processors. There are two types of IEEE floating-point formats (IEEE 754 standard). One is the *IEEE single precision format*, and the other is the *IEEE double precision format*. The single precision format is described in Figure 8.

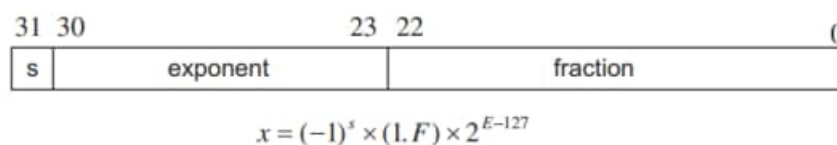


Figure 8: IEEE single precision floating point format.

- The format of IEEE single precision floating-point standard representation requires:
 - 23 fraction bits F ,
 - 8 exponent bits E ,
 - and 1 sign bit S ,
- A total of **32 bits** for each word.
- F is the mantissa in 2's complement positive binary fraction represented from bit 0 to bit 22.
- The mantissa is within the normalized range limits between $+1$ and $+2$.

- The sign bit S is employed to indicate the sign of the number, where when $S = 1$ the number is negative, and when $S = 0$ the number is positive.
- The exponent E is in **excess 127** form. The value of 127 is the offset from the 8-bit exponent range from 0 to 255, so that **E-127** will have a range from **-127 to +128**.
- The formula shown in Figure 8 can be applied to convert the IEEE 754 standard (single precision) to the decimal number.

The following simple examples also illustrate this conversion:

Use the formula $x = (-1)^S \times (1.F) \times 2^{E-127}$

0 10000000 000000000000000000000000 is in IEEE single precision format

Sign	Exponent	Fraction
0	10000000	000000000000000000000000
Negative	=128	= 0

$$= (-1)^0 \times (1.0_2) \times 2^{128-127} = 2.0 \text{ is the value in decimal}$$

Similarly,

$$0 \ 10000001 \ 101000000000000000000000 = (-1)^0 \times (1.101_2) \times 2^{129-127} = 6.5$$

$$1 \ 10000001 \ 101000000000000000000000 = (-1)^1 \times (1.101_2) \times 2^{129-127} = -6.5$$

EXAMPLE 9.11

Convert the following number in IEEE single precision format to decimal format:

110000000.010...0000

Solution:

From the bit pattern in Figure 9.11, we can identify the sign bit, exponent, and fractional as

$$s = 1, E = 2^7 = 128$$

$$1.F = 1.01_2 = (2)^0 + (2)^{-2} = 1.25$$

Then, applying the conversion formula leads to

$$x = (-1)^1 (1.25) \times 2^{128-127} = -1.25 \times 2^1 = -2.5$$

In conclusion, the value x represented by the word can be determined based on the following rules, including all the exceptional cases:

- If $E = 255$ and F is nonzero, then $x = NaN$ ("Not a number").
- If $E = 255$, F is zero, and S is 1, then $x = -\text{Infinity}$.
- If $E = 255$, F is zero, and S is 0, then $x = +\text{Infinity}$.
- If $0 < E < 255$, then $x = (-1)^S \times (1.F) \times 2^{E-127}$, where $1.F$ represents the binary number created by prefixing F with an implicit leading 1 and a binary point.
- If $E = 0$ and F is nonzero, then $x = (-1)^S \times (0.F) \times 2^{-126}$. This is an "unnormalized" value.
- If $E = 0$, F is zero, and S is 1, then $x = -0$.
- If $E = 0$, F is zero, and S is 0, then $x = 0$.

Typical and exceptional examples are shown below:

000000000 000000000000000000000000 = 0
 100000000 000000000000000000000000 = -0
 011111111 000000000000000000000000 = Infinity
 111111111 000000000000000000000000 = -Infinity
 011111111 000001000000000000000000 = NaN (Not a Number)
 111111111 001000100010010101010101 = NaN (Not a Number)
 000000001 000000000000000000000000 = $(-1)^0 \times (1.0_2) \times 2^{1-127} = 2^{-126}$
 000000000 100000000000000000000000 = $(-1)^0 \times (0.1_2) \times 2^{0-126} = 2^{-127}$
 000000000 100000000000000000000001 = $(-1)^0 \times (0.000000000000000000000001_2) \times 2^{0-126} = 2^{-149}$
 (smallest positive value)

2. Double Precision Format

The IEEE double precision format is described in Figure 9. The IEEE double precision floating-point standard representation requires a **64-bit word**, which may be numbered from 0 to 63, left to right. The first bit is the **sign bit S**, the next **eleven bits** are the **exponent bits E**, and the final **52 bits** are the **fraction bits F**. The IEEE floating-point format in double precision significantly **increases the dynamic range** of number representation since there are eleven exponent bits; the double-precision format also reduces the interval size in the mantissa normalized range of +1 to +2, since there are 52 mantissa bits as compare with the single precision case of 23 bits. Applying the conversion formula shown in Figure 9 is like the single precision case.

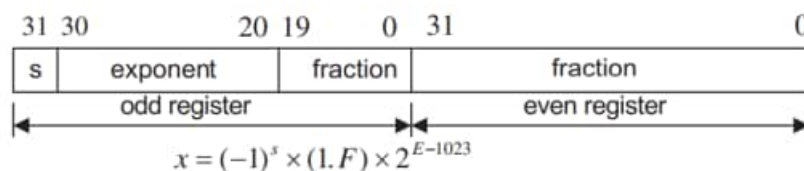


Figure 9: IEEE double precision floating point format.

EXAMPLE 9.12

Convert the following number in IEEE double precision format to the decimal format:

001000...0.110...000

Solution:

Using the bit pattern in Figure 9 , we have

$$s = 0, E = 2^9 = 512 \text{ and}$$

$$1.F = 1.11_2 = (2)^0 + (2)^{-1} + (2)^{-2} = 1.75$$

Then, applying the double precision formula yields

$$x = (-1)^0 (1.75) \times 2^{512-1023} = 1.75 \times 2^{-511} = 2.6104 \times 10^{-154}$$

For the purpose of completeness, rules for determining the value x represented by the double-precision word are listed as follows:

- If $E = 2,047$ and F is nonzero, then $x = NaN$ ("Not a number").
- If $E = 2,047$, F is zero, and S is 1, then $x = -\text{Infinity}$.
- If $E = 2,047$, F is zero, and S is 0, then $x = +\text{Infinity}$.
- If $0 < E < 2,047$, then $x = (-1)^s \times (1.F) \times 2^{E-1,023}$, where "1.F" is intended to represent the binary number created by prefixing F with an implicit leading 1 and a binary point.
- If $E = 0$ and F is nonzero, then $x = (-1)^s \times (0.F) \times 2^{-1022}$. This is an "unnormalized" value.
- If $E = 0$, F is zero, and S is 1, then $x = -0$.
- If $E = 0$, F is zero, and S is 0, then $x = 0$.

Fixed-Point Digital Signal Processors

- Analog Devices, Texas Instruments, and Motorola all manufacture **fixed-point** DS processors.
- Analog Devices offers a fixed-point DSP family such as the ADSP21xx.
- Texas Instruments provides various generations of fixed-point DS processors based on historical development, architecture features, and computational performance. Some of the most common ones are the TMS320C1x (first generation), TMS320C2x, TMS320C5x, and TMS320C62x.
- Motorola manufactures a variety of fixed-point processors, such as the DSP5600x family.
- The new families of fixed-point DS processors are expected to continue to grow.
- Since they share some basic common features such as program memory and data memory with associated address buses, arithmetic logic units (ALUs), program control units, MACs, shift units, and address generators, here we focus on an overview of the TMS320C54x processor.

The typical TMS320C54x fixed-point DSP architecture appears in Figure 10. The features are explained below:

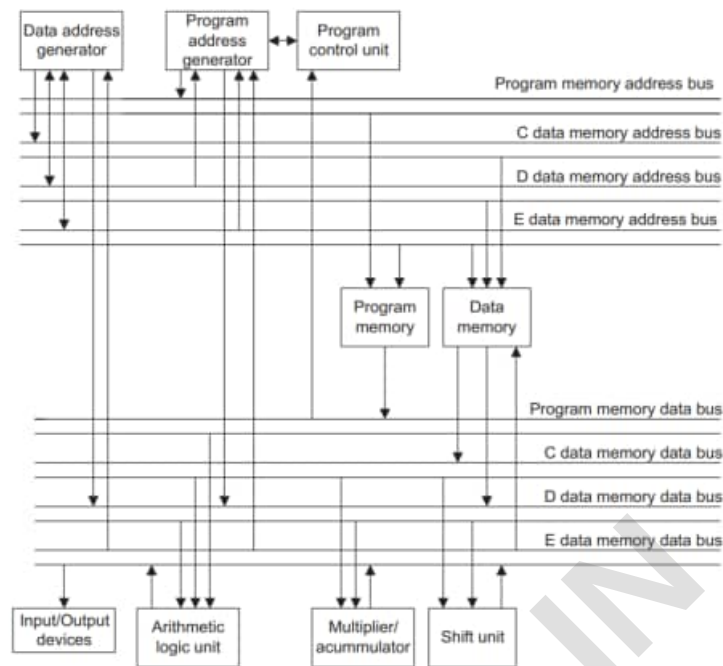


Figure 10: Basic architecture of the TMS320C54x family.

- The fixed-point TMS320C50 families supporting **16-bit data** have *on-chip program memory* and *data memory* in various sizes and configurations.
- They include **data RAM** (random access memory) and **program ROM** (read-only memory) used for program code, instruction, and data.
- **Four data buses** and **four address buses** are accommodated to work with the data memory and program memory. The program memory address bus and program memory data bus are responsible for fetching program instructions.
- As shown in Figure 10, the **C and D** data memory address buses and the C and D data memory data buses deal with **fetching data from the data memory (operands)** while the **E** data memory address bus and E data memory data bus are dedicated to **moving data into data memory (results)**. In addition, the E memory data bus can access the I/O devices.
- Computational units consist of an ALU, a MAC, and a shift unit. For the TMS320C54x family, the ALU can fetch data from the C, D, and program memory data buses and access the E memory data bus.
- It has *two independent 40-bit accumulators*, which are able to operate **40-bit addition**.
- The **multiplier**, which can fetch data from the C and D memory data buses and write data via the E data memory data bus, is capable of operating **17-bit 17-bit multiplications**.
- The **40-bit shifter** has the same capability of bus access as the MAC, allowing all possible *shifts for scaling and fractional arithmetic* such as those we have discussed for the Q-format.
- The **program control unit** *fetches instructions* via the program memory data bus.
- Again, in order to speed up memory access, there are **two address generators available**: one responsible for *program addresses* and one for *data addresses*.
- **Advanced Harvard architecture** is employed, where *several instructions operate at the same time* for a given single instruction cycle.

- **Processing performance offers 40 MIPS** (million instruction sets per second).

Floating-Point Processors

- Floating-point DS processors perform DSP operations using *floating-point arithmetic*, as we discussed before.
- The advantages of using the floating-point processor include getting rid of *finite word length effects* such as *overflows*, *round-off errors*, *truncation errors*, and *coefficient quantization error*.
- Hence, in terms of coding, we *do not need to scale input samples to avoid overflow*, *shift the accumulator result to fit the DAC word size*, *scale the filter coefficients*, or *apply Q-format arithmetic*.
- A floating-point DS processor with high speed and calculation precision *facilitates a friendly environment* to develop and implement DSP algorithms.
- Analog Devices provides floating-point DSP families such as ADSP210xx and TigerSHARC.
- Texas Instruments offers a wide range of the floating-point DSP families, in which the TMS320C3x is the first generation, followed by the TMS320C4x and TMS320C67x families.
- Since the first generation of a floating-point DS processor is less complicated than later generations but still has the common basic features, we review the first-generation architecture first.
- Figure 11 shows the typical architecture of Texas Instruments' TMS320C3x family of processors. We discuss some key features briefly.

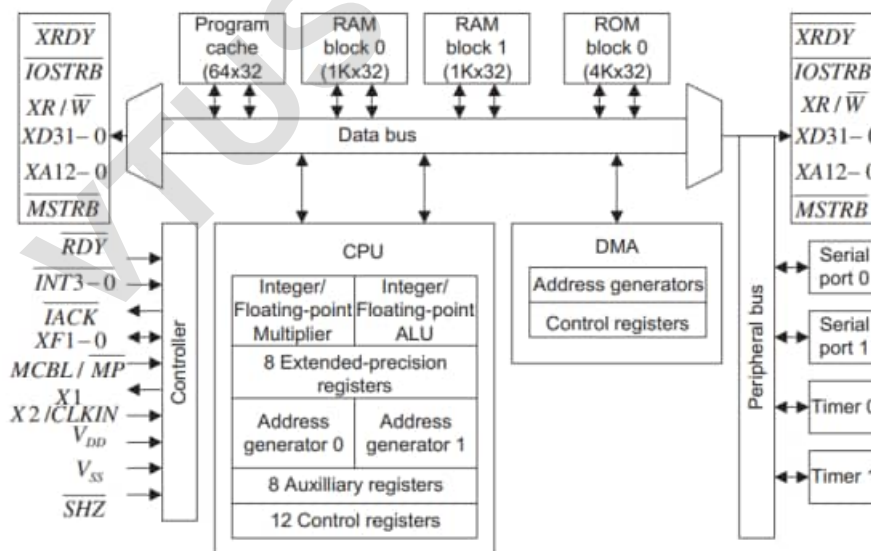


Figure 11: The typical TMS320C3x floating-point DS processor.

- The TMS320C3x family consists of **32-bit single chip floating-point processors** that *support both integer and floating-point operations*.
- The processor has a large memory space and is equipped with dual-access on-chip memories.
- A program cache is employed to enhance the execution of commonly used codes.

- Similar to the fixed-point processor, it uses the **Harvard architecture**, where there are separate buses used for program and data so that instructions can be fetched at the same time that data are being accessed.
- There also exist memory buses and data buses for **direct-memory access (DMA)** for concurrent I/O and CPU operations, and peripheral access such as serial ports, I/O ports, memory expansion, and an external clock.
- The C3x CPU contains the floating-point/integer multiplier; an **ALU**, which is capable of operating both integer and floating-point arithmetic; a **32-bit barrel shifter**; **internal buses**; a **CPU register file**; and dedicated **auxiliary register arithmetic units (ARAUs)**.
- The multiplier operates *single-cycle multiplications* on **24-bit integers** and on **32-bit floating-point** values.
- Using parallel instructions to perform a multiplication, an ALU will cost a **single cycle**, which means that *a multiplication and an addition are equally fast*.
- The ARAUs support addressing modes, in which some of them are specific to DSP such as circular buffering and bit-reversal addressing (digital filtering and FFT operations).
- The CPU register file offers **28 registers**, which can be operated on by the multiplier and ALU. The special functions of the registers include *eight-extended 40-bit precision registers* for maintaining accuracy of the floating-point results.
- Eight auxiliary registers can be used for addressing and for integer arithmetic.
- These registers provide internal temporary storage of internal variables instead of external memory storage, to allow performance of arithmetic between registers. In this way, program efficiency is greatly increased.
- The *prominent feature of C3x is its floating-point capability*, allowing operation of numbers with a very large dynamic range.
- It offers implementation of the DSP algorithm *without worrying about problems such as overflows and coefficient quantization*.
- *Three floating-point formats are supported.*
 - A short 16-bit floating-point format has 4 exponent bits, 1 sign bit, and 11 mantissa bits.
 - A 32-bit single precision format has 8 exponent bits, 1 sign bit, and 23 fraction bits.
 - A 40-bit extended precision format contains 8 exponent bits, 1 sign bit, and 31 fraction bits. Although the formats are slightly different from the IEEE 754 standard, conversions are available between these formats.
- The TMS320C30 offers **high-speed performance** with *60-nanosecond single-cycle instruction execution time, which is equivalent to 16.7 MIPS*.
- For **speech quality applications** with an *8 kHz sampling rate*, it can handle over *2,000 single-cycle instructions between two samples (125 microseconds)*.
- With instruction enhancements such as pipelines executing each instruction in a single cycle (four cycles required from fetch to execution by the instruction itself) and a multiple interrupt structure, this high-speed processor validates implementation of real-time applications in floating-point arithmetic.

FINITE IMPULSE RESPONSE AND INFINITE IMPULSE RESPONSE FILTER IMPLEMENTATIONS IN FIXED-POINT SYSTEMS

With knowledge of the IEEE format and of filter realization structures such as direct-form I, direct - form II, and parallel and cascade forms, we can study digital filter implementation in the fixed-point processor.

- In the fixed-point system, where only integer arithmetic is used, we prefer input data, filter coefficients, and processed output data to be in the **Q-format**. In this way, we avoid overflow due to multiplication and can prevent overflow due to addition by scaling input data.
- When the filter coefficients are **out of the Q-format range**, "**coefficient scaling**" must be considered to maintain the Q-format.
- We develop FIR filter implementation in Q-format first, and then infinite impulse response (IIR) filter implementation next.
- In addition, we assume that with a given input range in Q-format, the filter output is always in Q-format even if the filter passband gain is larger than 1.
- First, to avoid the overflow for an adder, we can scale the input down by a scale factor S, which can be safely determined by the following equation,

$$S = I_{max} \times \sum_{k=0}^{\infty} |h(k)| = I_{max} \times (|h(0)| + |h(1)| + |h(2)| + \dots) \quad (1)$$

where $h(k)$ is the impulse response of the adder output and I_{max} the maximum amplitude of the input in Q-format. Note that this is not an optimal factor in terms of reduced signal-to-noise ratio. However, it shall prevent the overflow. Equation (1) means that the adder output can actually be expressed as a convolution output:

$$\text{adder output} = h(0)x(n) + h(1)x(n-1) + h(2)x(n-2) + \dots$$

Assuming the **worst conditions**, that is, that all the inputs $x(n)$ reach a maximum value of I_{max} and all the impulse coefficients are positive, the sum of the adder gives the most conservative scale factor, as shown in Equation (1).

Hence, **scaling down the input by a factor of S** will guarantee that the output of the adder is in Q-format.

When some of the FIR coefficients are larger than 1, which is beyond the range of Q-format representation, **coefficient scaling is required**. The idea is that **scaling down the coefficients will make them less than 1**, and later the **filtered output will be scaled up by the same amount** before it is sent to DAC.

Figure 12 describes the modified implementation.

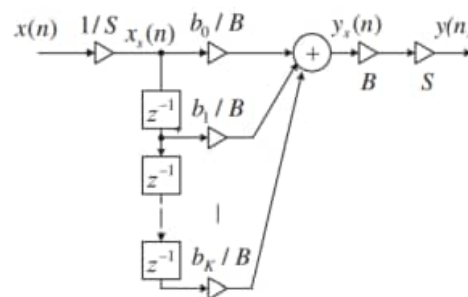


Figure 12: Direct-form I implementation of the FIR filter.

In the figure, the input is scaled down by S and the filter coefficients are scaled by B . The output is scaled up by multiplying with B and S to restore the value.

The scale factor B makes the coefficients $\mathbf{b}_k/B$ convertible to the Q-format. The scale factors S and B are usually chosen to be a power of 2, so the simple shift operation can be used in the coding process. Let us implement an FIR filter containing filter coefficients larger than 1 in the fixed-point implementation.

Example:

Given the FIR filter, $y(n) = 0.9x(n) + 3x(n-1) + 0.9x(n-2)$

with a passband gain of 4, and assuming that the input range only occupies one quarter of the full range for a particular application, develop the DSP implementation equations in the Q-15 fixed point system.

Solution: The adder may cause overflow if the input data exists for one quarter of the full dynamic range. The scale factor is determined using the impulse response, which consists of the FIR filter coefficients.

$$S = \frac{1}{4} (|h(0)| + |h(1)| + |h(2)|) = \frac{1}{4} (0.9 + 3 + 0.9) = 1.2$$

Overflow may occur. Hence, we select $S = 2$ (it is a power of 2). We choose $B = 4$ to scale all the coefficients to be less than 1, so the Q-15 format can be used. As per the figure 12, the developed difference equations are given by,

$$x_s(n) = x(n)/2$$

input is scaled by $S=2$. $x_s(n)$ is the scaled input.

$$y_s(n) = 0.225x_s(n) + 0.75x_s(n-1) + 0.225x_s(n-2)$$

Coefficients in the difference equation given in problem statement is divided by 4 to avoid overflow.

$$y(n) = 8y_s(n)$$

Multiplying by 8 to restore the value back.

Next, the direct-form I implementation of the IIR filter is illustrated in Figure 13. As shown in the figure, the purpose of the scale factor C is to scale down the original filter coefficients to the Q-format. The factor C is usually chosen to be a power of 2 for using a simple shift operation in DSP.

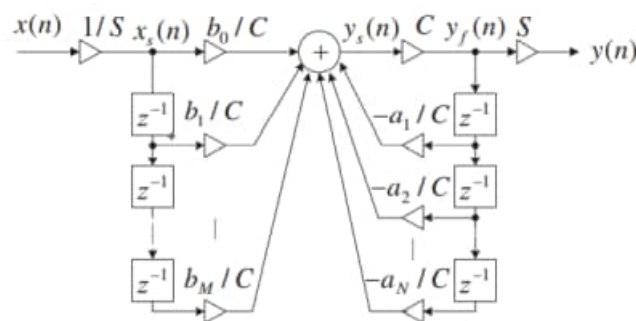


Figure 13: Direct-form I implementation of the IIR filter.

Example:

Given the IIR filter, $y(n) = 2x(n) + 0.5y(n-1)$

uses the direct-form I realization, for a particular application, the maximum input is $I_{\max}=0.010\dots0_2=0.25$. Develop the DSP implementation equations in the Q-15 fixed point system.

Solution: Given, difference equation, $y(n) = 2x(n) + 0.5y(n-1)$

The IIR filter has a transfer function is found by applying z-transform on both sides and rearranging,

$$H(z) = \frac{2}{1 - 0.5z^{-1}} = \frac{2z}{z - 0.5}$$

Applying the inverse z-transform, we obtain the impulse response.

$$h(n) = 2 \times (0.5)^n u(n)$$

To prevent overflow in the adder, we can compute the S factor with the help of the Maclaurin series or approximation equation (1) numerically. We get,

$$S = 0.25 \times [2(0.5)^0 + 2(0.5)^1 + 2(0.5)^2 + \dots] = 0.25 \times 2 \times \frac{1}{1 - 0.5} = 1$$

$$\text{Because, } \sum_{n=0}^{\infty} a^n = \frac{1}{1 - a}$$

Hence, we do not need to perform input scaling. However, we need scale down all the coefficients to use the Q-15 format. A factor of $C = 4$ is selected. From Figure 13, we get the difference equations as

$$x_s(n) = x(n)$$

$$y_s(n) = 0.5 x_s(n) + 0.125 y_f(n-1) \text{ [After scaling by } c=4]$$

Also, $y_f(n) = 4 y_s(n)$

and $y(n) = y_f(n)$

We can also develop these equations directly.

First, we divide the original difference equation by a factor of 4 to scale down all the coefficients to be less than 1, that is,

$$\frac{1}{4} y_f(n) = \frac{1}{4} \times 2x_s(n) + \frac{1}{4} \times 0.5 y_f(n-1)$$

then define a scaled output,

$$y_s(n) = \frac{1}{4} y_f(n)$$

Finally, substituting $y_s(n)$ on the left side of the scaled equation and rescaling up the filter output as $y_f(n) = 4y_s(n)$, we have the same result as before.

The fixed-point implementation for direct-form II is more complicated. The developed direct-form II implementation of the IIR filter is illustrated in Figure 14. As shown in the figure, two scale factors A and B are designated to scale denominator coefficients and numerator coefficients to their Q-format representations, respectively. Here S is a special factor to scale down the input sample so that the numerical overflow in the first sum in Figure 14. can be prevented. The difference equations describing the two stages are:

$$w(n) = x(n) - a_1 w(n-1) - a_2 w(n-2) - \dots - a_M w(n-M)$$

$$y(n) = b_0 w(n) + b_1 w(n-1) + \dots + b_M w(n-M)$$

The first equation is scaled down by the factor A to ensure that all the denominator coefficients are less than 1, that is,

$$w_s(n) = \frac{1}{A} w(n) = \frac{1}{A} x(n) - \frac{1}{A} a_1 w(n-1) - \frac{1}{A} a_2 w(n-2) - \dots - \frac{1}{A} a_M w(n-M)$$

$$w(n) = A \times w_s(n)$$

Similarly, the second equation yields,

$$y_s(n) = \frac{1}{B} y(n) = \frac{1}{B} b_0 w(n) + \frac{1}{B} b_1 w(n-1) + \dots + \frac{1}{B} b_M w(n-M)$$

and

$$y(n) = B \times y_s(n)$$

To avoid the first adder overflow (first equation), the scale factor S can be safely determined by Equation (1)

$$S = I_{max} \times (|h(0)| + |h(1)| + |h(2)| + \dots) \quad (1)$$

where $h(k)$ is the impulse response due to the denominator polynomial of the IIR filter, where the poles can cause a large value to the first sum. Hence, $h(k)$ is given by,

$$h(n) = z^{-1} \left(\frac{1}{1 + a_1 z^{-1} + \dots + a_M z^{-M}} \right) \quad (2)$$

All the scale factors A, B, and S are usually chosen to a power of a power of 2, respectively, so that the shift operations can be used in the coding process.

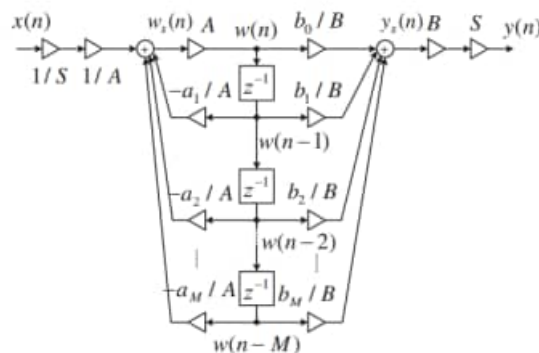


Figure 14: Direct-form II implementation of the IIR filter.

Example:

Given the IIR filter,

$$y(n) = 0.75x(n) + 1.49x(n-1) + 0.75x(n-2) - 1.52y(n-1) - 0.64y(n-2)$$

with a passband gain of 1 and a full range of input, use the direct-form II implementation to develop the DSP implementation in the Q-15 fixed-point system.

Solution: The difference equations without scaling in the direct-form II implementation are given by,

$$w(n) = x(n) - 1.52 w(n-1) - 0.64 w(n-2)$$

$$y(n) = 0.75 w(n) + 1.49 w(n-1) + 0.75 w(n-2)$$

To prevent overflow in the first adder, we obtain the reciprocal of the denominator polynomial as

$$A(z) = \frac{1}{1 + 1.52 z^{-1} + 0.64 z^{-2}}$$

We choose the S factor as $S = 16$ and we choose $A=2$ to scale down the denominator coefficients by half.

Since the second adder output after scaling is

$$y_s(n) = \frac{0.75}{B} w(n) + \frac{1.49}{B} w(n-1) + \frac{0.75}{B} w(n-2)$$

to avoid second adder overflow we have to ensure that each coefficient is less than 1, along with the sum of the absolute values:

$$\frac{0.75}{B} + \frac{1.49}{B} + \frac{0.75}{B} < 1$$

Hence $B = 4$ is selected. We develop the DSP equations as

$$x_s(n) = x(n)/16$$

$$w_s(n) = 0.5 x_s(n) - 0.76 w(n-1) - 0.32 w(n-2)$$

$$w(n) = 2 w_s(n)$$

$$y_s(n) = 0.1875 w(n) + 0.3725 w(n-1) + 0.1875 w(n-2)$$

$$y(n) = (B \times S) y_s(n) = 64 y_s(n)$$

VTU Model Questions

Module – 5			
Q.9	(a)	With the block diagram explain Digital signal processors based on the Harvard architecture.	6
	(b)	Discuss briefly the following special Digital Signal Processor hardware units. (i) Multiplier and Accumulator (MAC) Unit (ii) Shifters (iii) Address Generators	(3+2+5) 10
	(c)	Convert the following decimal numbers into Q – 15 representation. (i) –0.1958 (ii) 0.560123	4
OR			
Q.10	(a)	Given the FIR filter $y(n) = 0.9 x(n) + 3 x(n - 1) - 0.9x(n - 2)$ With the passband gain of 4, and assuming that the input range occupies only $\frac{1}{4}$ of the full range for a particular application. Develop the DSP implementation equations in the Q – 15 fixed – point system.	6
	(b)	Discuss the following IEEE Floating – Point Formats. (i) Single Precision Format (ii) Double Precision Format	6
	(c)	With the diagram explain the basic architecture of TMS320C54x family processor.	8